Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 2

з дисципліни «Системне програмування» на тему:

«Інтеграція сенсорної підсистеми: схема підключення, верифікація інтерфейсів логічним аналізатором, архітектура програмного проєкту»

Виконали:
студенти групи ІО-32
Крадожон М. Р.
Пилипчук В. Д.
студенти групи ІО-35
Комар В. О.
Сабадаш В. Г.
Хімко А. Р.

Перевірив:
Гордієнко Н. Ю.

Київ — 2026 р.

Мета роботи:

Мета лабораторної роботи полягала в забезпеченні апаратної та програмної сумісність компонентів мультирежимного реєстратора інцидентів на базі платформи STM32 і підготовці основи для подальшої розробки його функціональності. На початковому етапі основну увагу було зосереджено на проєктуванні схеми підключення сенсорів і виконавчих пристроїв, виборі відповідних виводів мікроконтролера, перевірці сумісності периферійних інтерфейсів і розрахунку бюджету живлення. Важливим завданням було забезпечити одночасну роботу всіх передбачених проєктом компонентів, а саме: інерціального сенсорного модуля GY-89, камери OV7670 і зовнішньої flash-пам’яті W25Q64.

У процесі виконання лабораторної роботи мета розширилася до практичної перевірки роботи апаратних інтерфейсів за допомогою логічного аналізатора та зіставлення отриманих сигналів із документацією виробників. Окрему увагу приділено організації програмного проєкту: розмежуванню апаратно-залежного коду, драйверів пристроїв і прикладної логіки, налаштуванню відтворюваної системи збірки та підготовці автоматизованих тестів, які можна виконувати без фізичної плати.

Розподіл обов’язків у команді:

Таблиця 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Владислав Комар	Системний аналітик і технічний письменник	1. Опрацьовано datasheet STM32F411RE, RM0383, UM1724 і документацію сенсорів та модулів. 2. Розроблено pinout із AF, контактами CN7/CN10, адресами ID-регістрів, живленням, підтяжками та параметрами шин. 3. Перевірено конфлікти пінів, периферії й DMA; визначено альтернативи та наведено аргументацію до ухвалених рішень.	docs/pinout.md docs/reports/report_2.pdf	PR #6 PR #8 f482875

Продовження таблиці 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Владислав Комар	Системний аналітик і технічний письменник	4. Підготовлено фінальний звіт: сформовано структуру, зібрано матеріали від учасників команди, оформлено титульний аркуш, описано процес виконання та проблеми, підготовлено фінальну версію у форматі PDF.	docs/pinout.md docs/reports/ report_2.pdf	PR #6 PR #8 f482875
Вікторія Сабадаш	Інженер з апаратного забезпечення	1. Зібрано стенд за погодженим pinout і перевірено фактичну відповідність монтажу таблиці. 2. Проведено первинні вимірювання живлення та перевірку рівнів сигналів. 3. Перевірено присутність пристроїв, отримання ID і правдоподібність даних у спільній прошивці. 4. Підготовлено фото стенда, UART-лог самоперевірки та відтворювані записи інтерфейсів.	docs/captures/*.sr	2378c37*
Максим Крадожон	Аналітик протоколів	1. Декодовано трафік шин I2C, SPI, SCCB та UART у PulseView та sigrok-cli. 2. Зіставлено вичитані ID пристроїв (LSM303D, L3GD20H, BMP180, W25Q64JV, OV7670) із первинними datasheet. 3. Проведено часові вимірювання (частоти, періоди, setup/hold таймінги, відсутність міжбайтових затримок). 4. Досліджено поведінку 9-го такту SCCB та аномалію Framing Error в UART через перехідні процеси та фазовий дрейф. 5. Сформовано тестові вектори для хостових unit-тестів (контракт 4 → 6).	docs/captures/ analysis.md docs/captures/ annotations/*.txt docs/lab_2/img *.png	2378c37
Вікторія Пилипчук	Архітектор вбудованого ПЗ	1. Організовано програму за шарами: згенерований код, Platform, Devices та App. 2. Узгоджено абстракції шин, щоб драйвери не залежали безпосередньо від HAL. 3. Підключено потрібні модулі HAL/CMSIS, налаштовано build і перевірку напрямку залежностей. 4. Налаштовано/оновлено CI для запуску збірки та тестів на pull request.	firmware/*	c7153ca 0bb5701 5f5c42a 3750979 30d05d1

Кінець таблиці 1

Учасник/ця	Роль	Внесок	Артефакти	Коміти / PR
Артем Хімко	Інженер з тестування	1. Налаштовано хостові тести без плати і незалежний від HAL запуск логіки bringup.c. 2. Створено test doubles для I²C, SPI-flash W25Q64, SCCB і UART. 3. Написано 35 тестів (216 перевірок): перетворення даних сенсорів, помилки шин (NACK, збій), послідовність команд SPI-flash, UART-команди. 4. Використано реальні байти з записів логічного аналізатора як тестові вектори. 5. Налаштовано вимірювання покриття і перевірку порогу ≥ 80 % / ≥ 70 % у CI. 6. Додано job test у GitHub Actions на кожен pull request	/test_bringup.c, tests/fakes/fake_bus.c, tests/fakes/fake_bus.h, tests/minitest.h, tests/Makefile, tests/coverage_gate.sh,, .github/workflows/ci.yml (job test), docs/test-coverage.md	PR #7

* Записи сигналів логічного аналізатора (файли docs/captures/*.sr) виконувала Вікторія Сабадаш, але через відсутність електропостачання вони були передані Максиму Крадожону не через GitHub. Максим відправив у репозиторій файли docs/captures/*.sr (частина коміту 2378c37) від імені Вікторії. Це було зроблено з метою уникнення блокування контракту, оскільки без файлів docs/captures/*.sr Максим як аналітик протоколів не міг розпочати виконання своєї частини роботи.

Виконання лабораторної роботи:

1. Проєктування та обґрунтування схеми підключення

Апаратну конфігурацію розроблено для плати STM32 NUCLEO-F411RE на базі STM32F411RE. Мета розпіновки — забезпечити одночасну роботу сенсорної підсистеми, камери, зовнішньої пам’яті та командного інтерфейсу, не порушуючи роботи штатних засобів програмування й налагодження Nucleo. Під час вибору контактів враховано alternate functions мікроконтролера, доступність контактів на роз’ємах плати, маршрути DMA та вимоги до рівнів сигналів.

Повна таблиця всіх з’єднань із номерами AF, контактами CN7/CN10, рівнями сигналів, місцями підтяжок, обґрунтуванням вибору кожного піна, детальним розрахунком частот, живлення та схемою монтажу наведена в репозиторії у файлі docs/pinout.md. Цей розділ звіту є стислим викладом зазначеного документа.

1.1. Схема підключення та pinout-таблиця

1.1.1. Склад апаратної конфігурації та принципи розпіновки

Стенд складається з: мікроконтролера STM32 NUCLEO-F411RE, модуля GY-89 (LSM303D, L3GD20H і BMP180), камери OV7670, SPI Flash W25Q64 та USB-UART адаптера YP-05 на FT232RL. Зовнішні модулі під’єднуються до плати без пайки через штатні роз’єми CN7 і CN10. Всі перелічені пристрої мають працювати в одній прошивці, а не в окремих тестових конфігураціях.

Таблиця 2

Пристрій	Інтерфейс	Призначення в проєкті	Живлення
GY-89 (LSM303D, L3GD20H, BMP180)	I2C1	Прискорення, кутова швидкість, магнітне поле, тиск і температура	VIN = 5 В за документацією використаного модуля; GND спільний
OV7670	SCCB + паралельний DVP	Налаштування камери та отримання кадру під час інциденту	3,3 В; GND спільний
W25Q64	SPI2	Енергонезалежний журнал подій і збереження даних/кадрів	3,3 В; GND спільний
YP-05 на FT232RL	USART1 ↔ USB	Командний інтерфейс із ПК	Окреме USB-живлення від ПК; VCC до Nucleo не підключати

Для сенсорного модуля виділено апаратний I2C1 на PB8/PB9 (AF4), для пам’яті — SPI2 на PB13/PB14/PB15 (AF5), а для командного порту — USART1 на PA9/PA10 (AF7). OV7670 використовує окрему програмно реалізовану SCCB на PB6/PB7, оскільки STM32F411RE не має периферійного блока DCMI. Паралельна шина даних камери розміщена на GPIOC, щоб рівні восьми бітів можна було зчитувати одним читанням GPIOC->IDR. Такт XCLK формує TIM1_CH1 на PA8, а вхід PCLK підведено до TIM3_CH4 на PC9.

1.1.2. Основні з’єднання модулів

Для стислого подання у звіті наведено основні сигнальні з’єднання. Повна pinout-таблиця з AF, контактами роз’ємів, напрямком сигналу, рівнями напруг, підтяжками та обґрунтуванням кожного виводу зберігається в окремому документі docs/pinout.md.

Таблиця 3

Модуль / сигнал	Пін STM32 та функція	Контакт Nucleo	Підключення до модуля
GY-89 SCL	PB8, AF4, I2C1_SCL	CN10-3 / D15	SCL
GY-89 SDA	PB9, AF4, I2C1_SDA	CN10-5 / D14	SDA
GY-89 живлення	+5V; GND	CN7-18; CN7-8	VIN; GND

Кінець таблиці 3

Модуль / сигнал	Пін STM32 та функція	Контакт Nucleo	Підключення до модуля
W25Q64 SCK / MISO / MOSI	PB13 / PB14 / PB15, AF5, SPI2	CN10-30 / CN10-28 / CN10-26	CLK / D0 (MISO) / DI (MOSI)
W25Q64 CS#	PB12, GPIO	CN10-16	CS; програмне керування рівнем
W25Q64 живлення	+3V3; GND	CN7-16; CN7-8	VCC; GND
YP-05 UART TX/RX	PA9, AF7, USART1_TX; PA10, AF7, USART1_RX	CN10-21 / CN10-33 (D8 / D2)	PA9 → RXD; TXD → PA10
YP-05 земля	GND	CN7-22	GND; VCC адаптера до Nucleo не підключати
OV7670 SCCB	PB6 / PB7, GPIO	CN10-17 / CN7-21	SCL/SIOC / SDA/SIOD
OV7670 тактування й керування	PA8, TIM1_CH1 AF1; PA0; PA1	CN10-23 (D7); CN7-28 (A0); CN7-30 (A1)	XCLK; RESET; PWDN
OV7670 DVP data	PC2, PC3, PC4, PC5, PC6, PC7, PC8, PC10, GPIO	CN7-35; CN7-37; CN10-34; CN10-6; CN10-4; CN10-19; CN10-2; CN7-1	D0, D1, D2, D3, D4, D5, D6, D7 відповідно
OV7670 синхронізація	PC9, TIM3_CH4 AF2; PC11; PC12	CN10-1; CN7-2; CN7-3	PCLK; HREF; VSYNC
OV7670 живлення	+3V3; GND	CN7-16; CN7-8	3.3V; DGND

1.1.3. Основні з’єднання модулів

1) Перед монтажем від’єднати USB-живлення Nucleo та окреме USB-підключення адаптера YP-05.

2) Підключити GY-89 до PB8/PB9, +5V (VIN) і GND. Внутрішні підтяжки STM32 не використовувати як основні підтяжки I²C; на лініях мають бути коректні підтягувальні резистори модуля/шини.
3) Підключити W25Q64 до SPI2 (PB13/PB14/PB15), CS# — до PB12, живлення — до +3V3 і GND.
4) Під'єднати YP-05 перехресно: PA9/TX до RXD адаптера, TXD адаптера до PA10/RX, GND до GND. Не з'єднувати VCC адаптера з живленням Nucleo.
5) Під'єднати OV7670: спочатку SCCB, XCLK, RESET і PWDN; потім D0–D7, PCLK, HREF і VSYNC; наприкінці під'єднати 3,3 В та GND. Особливо перевірити, що D7 підключено до PC10, а PCLK — до PC9.
6) Перед увімкненням перевірити спільну землю, відсутність короткого замикання між живленням і GND, а також відповідність напруги кожного модуля документації.
7) Після подачі живлення виконати стартову ідентифікацію сенсорів, камери та Flash; до захоплення кадру переконатися, що на XCLK і PCLK є сигнал, а SCCB відповідає на запити.

1.2. Параметри інтерфейсів, адресація та живлення

1.2.1. Адреси та ідентифікатори пристроїв

На шині I2C1 використовуються три адреси, що не перетинаються. У прошивці доцільно зберігати 7-бітні адреси, а під час викликів STM32 HAL передавати значення addr7 << 1. Для SCCB адресні байти камери у документації подані з урахуванням біта R/W. Для W25Q64 адреса I²C не застосовується: ідентифікація виконується SPI-командою JEDEC ID.

Таблиця 4

Пристрій	Адреса / команда	Регістр або поле ID	Очікуване значення
LSM303D	I²C 7-bit 0x1D	WHO_AM_I, 0x0F	0x49
L3GD20H	I²C 7-bit 0x6B	WHO_AM_I, 0x0F	0xD7
BMP180	I²C 7-bit 0x77	CHIP_ID, 0xD0	0x55

Пристрій	Адреса / команда	Регістр або поле ID	Очікуване значення
OV7670	SCCB 7-bit 0x21; байти запис/читання 0x42/0x43	PID, 0x0A; VER, 0x0B	PID = 0x76; VER = 0x70
W25Q64FV	SPI-команда 0x9F	JEDEC ID: Manufacturer / Type / Capacity	EF 40 17
YP-05 / FT232RL	USART1, 115200 baud	Ідентифікація через регістр не передбачена	Перевірка двостороннім UART-обміном

1.2.2. Тактування та налаштування шин

Прийняте дерево тактування має SYSCLK = HCLK = 100 MHz, PCLK1 = 50 MHz і PCLK2 = 100 MHz. За APB1 prescaler = 2 таймери APB1 отримують 100 MHz; TIM1 на APB2 також працює від 100 MHz.

Таблиця 5

Інтерфейс	Піни / джерело тактування	Режим і розрахункові параметри
I2C1 для GY-89	PB8/PB9, AF4; PCLK1 = 50 MHz	Standard-mode, 100 kHz; CCR = 250, TRISE = 51, FREQ = 50 MHz.
SCCB для OV7670	PB6/PB7, GPIO;програмні затримки від HCLK = 100 MHz	Ціль — до 100 kHz; початкова тривалість HIGH/LOW близько 5 мкс. Апаратний prescaler відсутній.
SPI2 для W25Q64	PB13/PB14/PB15, AF5; PCLK1 = 50 MHz; CS# = PB12	Mode 0, CPOL = 0, CPHA = 0, 8 біт, MSB first. Дільник /16: 50 MHz / 16 = 3,125 MHz.
USART1 для YP-05	PA9/PA10, AF7; PCLK2 = 100 MHz	115200 baud, 8N1, oversampling 16; BRR = 0x0364, похибка приблизно 0,006 %.
USART2 monitor	PA2/PA3, ST-LINK VCP; PCLK1 = 50 MHz	115200 baud, 8N1; канал залишено для стандартного monitor.
Camera XCLK	PA8/TIM1_CH1, AF1; TIM1CLK = 100 MHz	10 MHz PWM: PSC = 0, ARR = 9, CCR1 = 5, заповнення близько 50 %.
Camera PCLK/DVP capture	PC9/TIM3_CH4, AF2; GPIOC->IDR	Цільова частота PCLK — орієнтовно до 1 MHz після налаштування регістрів камери; захоплення через DMA1 Stream 2 / Channel 5. Потрібна експериментальна перевірка.

Параметри I²C і SPI розраховані від частоти APB1: для I²C у Standard-mode CCR = 250; для SPI дільник /16 дає 3,125 MHz. На SCCB prescaler відсутній, оскільки передавання реалізується програмно. PCLK камери не задається таймером STM32: він залежить від XCLK і конфігурації внутрішнього тактування / дільника OV7670, тому має бути виміряний на PC9 після налаштування регістрів.

1.2.3. Бюджет живлення

GY-89 під’єднано до лінії +5V через VIN відповідно до документації використаного готового модуля; OV7670 і W25Q64 живляться від +3V3 Nucleo. YP-05 живиться окремо через власне USB Type-C з’єднання з ПК; його VCC не з’єднується з 3,3 В або 5 В плати. Спільний GND потрібен для належних логічних рівнів усіх сигнальних інтерфейсів.

Таблиця 6

Навантаження	**Джерело**	**Струм для попереднього розрахунку**	**Статус значення**
GY-89	+5V Nucleo → VIN	25 мА	Проєктний резерв; струм готового модуля слід виміряти.
OV7670	+3V3 Nucleo	30 мА	Проєктний резерв для активної камери.
W25Q64FV	+3V3 Nucleo	до 25 мА	Піковий бюджет для програмування/стирання за використаним даташитом.
YP-05 / FT232RL	Окремий USB порт ПК	25 мА	Проєктний резерв для готового адаптера; до бюджету Nucleo не додається.

Для зовнішніх навантажень лінії +3V3 прийнято 30 мА для OV7670 і 25 мА для W25Q64, тобто $I_{external}$ = 55 мА. У документації NUCLEO-F411RE для регулятора U4 зазначена максимальна струмова здатність до 500 мА; арифметична різниця становить 500 − 55 = 445 мА. Це не є гарантованим вільним запасом, оскільки потрібно відняти споживання самої Nucleo та врахувати температурні й USB-обмеження.

У розрахунку USB-живлення плати для конфігурації JP1 = OFF застосовано ліміт до 300 мА для плати й розширень. Попередній бюджет зовнішніх модулів становить 55 мА на +3V3 та 25 мА на +5V, разом 80 мА; арифметичний залишок до ліміту — 220 мА до врахування бортового споживання Nucleo. Положення JP1 і фактичний струм потрібно перевірити на стенді, а напругу +3V3 контролювати під час одночасного захоплення кадру та запису у Flash. За відсутності сервоприводів чи інших силових навантажень окреме зовнішнє джерело для сенсорів попередньо не потрібне, але це рішення остаточно підтверджується вимірюваннями.

1.3. Перевірка конфліктів і відхилені варіанти

1.3.1. Перевірка сумісності ресурсів

Під час проєктування перевірялися не лише повтори номерів GPIO, а й сумісність периферійних блоків, DMA, штатної обв’язки Nucleo та електричних рівнів. Основні рішення зведено нижче.

Таблиця 7

Тип конфлікту	Перевірка та ухвалене рішення
GPIO та alternate functions	Кожному зовнішньому сигналу призначено окремий фізичний пін. I2C1 закріплено за PB8/PB9; SCCB камери — за PB6/PB7; SPI2 — за PB13–PB15 і окремим CS# PB12; USART1 — за PA9/PA10.
Паралельний інтерфейс OV7670	STM32F411RE не має DCMI, тому DVP використовує GPIOC: D0–D6 на PC2–PC8, D7 на PC10, HREF на PC11, VSYNC на PC12. PCLK підключено до PC9/TIM3_CH4. Одне читання GPIOC->IDR дозволяє отримати зразок усіх ліній порту; працездатність такого механізму та максимальну частоту потрібно підтвердити вимірюваннями.
Таймери	TIM1_CH1 на PA8 генерує XCLK. TIM3_CH4 на PC9 використовується для події захоплення PCLK. Інші канали TIM3 не задіяні в цій конфігурації, отже їхні alternate functions на PC7/PC8 залишаються GPIO.
DMA	Для захоплення PCLK заплановано DMA1 Stream 2 / Channel 5. SPI2_TX використовує DMA1 Stream 4 / Channel 0; варіант із TIM3_CH1 відхилено, оскільки цей запит також маршрутизується на Stream 4, хоча й на інший Channel. Один DMA stream не може одночасно мати дві конфігурації CHSEL.
Бортові ресурси Nucleo	PA2/PA3 залишено за USART2 і ST-LINK VCP; PA13/PA14 — за SWD; PA5 — штатний LD2; PC13 — USER button. Для зовнішніх модулів ці піни не використовуються, а solder bridges плати змінювати не планується.
Електричні рівні та земля	OV7670 і W25Q64 живляться від 3,3 В; GY-89 — через VIN 5 В відповідно до документації готового модуля. SCCB/I²C працюють з open-drain і зовнішніми підтяжками. Земля Nucleo спільна з усіма модулями. VCC YP-05 не з’єднується з живленням Nucleo, щоб уникнути зворотного живлення.
Адресація	Пристрої GY-89 мають різні 7-бітні адреси 0x1D, 0x6B та 0x77. Камера SCCB фізично винесена на іншу пару GPIO та має адресу 0x21 (байти 0x42/0x43), тому адресного або лінійного конфлікту з I2C1 немає.

1.3.2. Відхилені варіанти розпіновки

Таблиця 8

Альтернативний варіант	Причина відмови
I2C1 для GY-89 на PB6/PB7	Ці самі фізичні контакти виділено для програмної SCCB камери. Спільне використання створило б конфлікт ліній між сенсорною шиною та керуванням OV7670. Тому I2C1 залишено на PB8/PB9.
SPI1 на PA5/PA6/PA7 для W25Q64	PA5 пов’язаний зі штатним світлодіодом LD2 (Arduino D13). Тактовий сигнал SPI перемикав би LED і заважав би його використанню як індикатора стану. Обрано SPI2 на PB13/PB14/PB15 із CS# на PB12.
YP-05 на USART2, PA2/PA3	Ця пара штатно з’єднана зі ST-LINK VCP. Підключення другого UART-моста без зміни плати створило б конфлікт із вбудованим каналом monitor. Тому YP-05 використовує USART1 на PA9/PA10, а USART2 залишається для ST-LINK.

Альтернативний варіант	Причина відмови
PCLK камери на PA6/TIM3_CH1	TIM3_CH1 маршрутизується на DMA1 Stream 4 / Channel 5, тоді як SPI2_TX використовує DMA1 Stream 4 / Channel 0. Оскільки один stream налаштовується лише на один канал вибору запиту за раз, одночасне використання створило б DMA-конфлікт. Для PCLK обрано PC9/TIM3_CH4 → DMA1 Stream 2 / Channel 5.

Отже, обрана розпіновка розділяє сигнали сенсорів, камери, SPI Flash і UART, зберігає штатний ST-LINK VCP та SWD і не потребує перепаювання solder bridges Nucleo. Найбільш ресурсомістким рішенням залишається захоплення паралельної шини камери без DCMI; його обмеження щодо швидкості та стабільності мають бути перевірені окремо під час запису PCLK, HREF, VSYNC і тестового кадру.

2. Монтаж стенда та перевірка працездатності пристроїв

Зібраний стенд наведено на рис. 1.

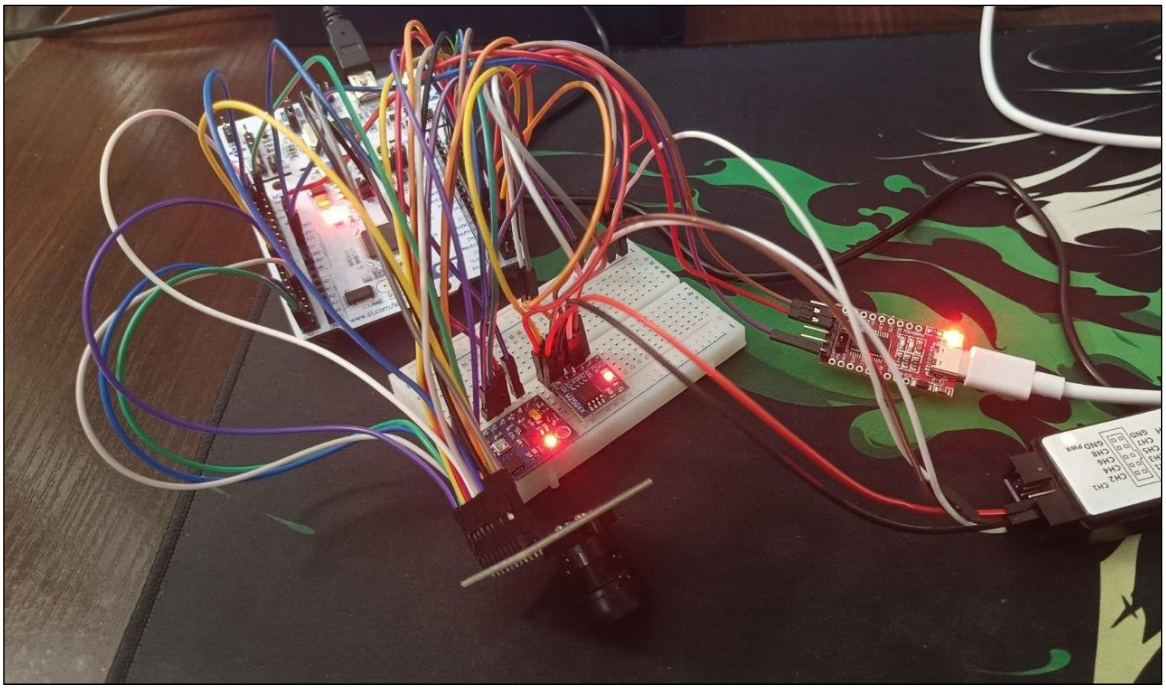


Рисунок 1

Нижче представлені результати режиму self-test, який виводить у UART зведення стану кожного пристрою.

```
=== SELFTEST ===
[I2C] LSM303D id=0x49 x=-10 y=1998 z=1047 mg -> OK
[I2C] L3GD20H id=0xD4 x=-1058 y=3176 z=1093 mdps -> OK
```

```
[I2C] BMP180 id=0x55 temp_x10=230 pressure=99944 Pa -> OK
[SPI] W25Q64 jedec=EF 40 17 -> OK
[SCCB] OV7670 pid=0x76 ver=0x73 write_ack=1 -> OK
=== END (5/5 OK) ===
```

У табл. 9 зібрані параметри захоплення.

Таблиця 9

№	Команда	Файл	Піни	Семплів	Частота	Тригер
1	i2c	01_i2c_whoami.sr	PB8 (SCL), PB9 (SDA)	100 000	2 MHz	падіння PB9
2	data	02_i2c_data_bmp180.sr	PB8, PB9	100 000	2 MHz	падіння PB9
3	spi	03_spi_jedec.sr	PB13 (SCK), PB15 (MOSI), PB14 (MISO), PB12 (CS#)	2 000 000	2 MHz	падіння PB12
4	flash	04_spi_flash.sr	PB13, PB15, PB14, PB12	2 000 000	2 MHz	падіння PB12
5	sccb	05_sccb_read_pid.sr	PB6 (SCL), PB7 (SDA)	100 000	1 MHz	падіння PB7
6	id	06_uart_selftest.sr	PA9	100 000	1 MHz	падіння PA9
7	uart	07_uart_hello_usart1.sr	PA9 (TX), PA10 (RX)	100 000	1 MHz	падіння PA9

3. Верифікація інтерфейсів логічним аналізатором

3.1 Записи та декодування транзакцій

Для апаратного аналізу використано 8-канальний логічний аналізатор (Saleae-сумісна апаратна частина на базі Cypress FX2LP, драйвер fx2lafw) під керуванням інструментів PulseView та sigrok-cli. Усі захоплені пакети збережено у форматі сесій .sr у каталозі docs/captures/.

3.1.1. Шина I²C1: Ідентифікація сенсорів та багатобайтове читання (01_i2c_whoami.sr)

- **LSM303D (Акселерометр/магнітометр):**

- Пряме читання з адреси 0x1D (8-бітний Read ID = 0x3B). Отримано значення регістра 0x0F (WHO_AM_I) = **0x49**, що повністю збігається з документацією чіпа LSM303D.
- Багатобайтове читання вихідних осей: передано команду суб-адреси 0xA8 (адреса регістра OUT_X_L_A з установленим бітом автоінкременту 0x80). Послідовно вичитано 6 байтів: 3F FE F8 7F 9B 43 (осі X, Y, Z).
- **L3GD20H (Гіроскоп):**
- Транзакція з повторним стартом (Repeated START): запис адреси 0x6B (W) → передача суб-адреси 0x0F → Sr → читання 0x6B (R). Отримано байт ідентифікатора **0xD4**, що підтверджує присутність L3GD20H.
- **BMP180 (Барометр):**
- Запис адреси 0x77 (W) → субадреса регістра 0xD0 (chip_id) → Sr → читання 0x77 (R). Отримано ідентифікатор **0x55**, що відповідає специфікації Bosch Sensortec.

3.1.2. Шина I²C1: Калібрування та робоче опитування BMP180 (02_i2c_data_bmp180.sr)

- **Читання EEPROM калібрування:** комбінована транзакція читання з адреси 0x77 починаючи з регістру 0xAA. Послідовно вичитано 22 байти калібрувальної матриці:

```
19 9C FB 72 C6 8E 85 F0 62 60 34 0D 19 73 00 2D 80 00 D1 F6 0A 94
```

- **Зчитування некомпенсованої температури (*UT*):** запис команди 0x2E у контрольний регістр 0xF4, очікування завершення конверсії, зчитування 2 байтів із регістрів 0xF6–0xF7: UT = 0x54BB (21691).

- **Зчитування некомпенсованого тиску (*UP*):** запис команди 0x34 (режим OSS = 0) у регістр 0xF4, очікування конверсії, зчитування 3 байтів із регістрів 0xF6–0xF8: UP = 0x9FD600 (40918).
- **Верифікація фізичної достовірності:** за формулами компенсації Bosch розраховано:
- Температура: **23.7 °C**.
- Атмосферний тиск: **998.4 гПа** (748.8 мм рт. ст.). Показники відповідають реальним умовам лабораторії.

3.1.3 Шина SCCB: Ідентифікація камери OmniVision OV7670 (05_sccb_read_pid.sr)

- Декодовано двофазні цикли звернення до камери за адресою 0x21 (8-бітні коди: запис 0x42, читання 0x43):
- Регістр 0x0A (PID MSB): зчитано **0x76**.
- Регістр 0x0B (VER LSB): зчитано **0x73**.
- Сумарний ідентифікатор 0x7673 відповідає сенсору OV7670 v1.4.
- **Поведінка дев'ятого такту:** зафіксовано ключову відмінність SCCB від I²C. У фазі читання на 9-му такті лінії SIOC мікроконтролер утримує лінію SIOD на високому рівні (стан NA / Don't Care), що стандартні декодери сприймають як NACK, проте це є штатним протокольним завершенням прийому в специфікації OmniVision. Транзакції розділяються безумовною умовою STOP без використання Repeated START.

3.1.4. Шина SPI2: Flash-пам'ять Winbond W25Q64JV (03_spi_jedec.sr, 04_spi_flash.sr)

- **Читання JEDEC ID (03_spi_jedec.sr):**

- При активному низькому рівні CS# передано команду 0x9F. На лінії MISO отримано 3 байти відповіді: **EF 40 17** (Manufacturer: Winbond (0xEF), Memory Type: 0x40, Capacity: 64 Mbit (0x17)).

- **Цикл запису та верифікації (04_spi_flash.sr):**

1. *Дозвіл запису:* передано обов'язкову однобайтову команду 0x06 (WREN).
2. *Програмування сторінки:* команда 0x02 (Page Program) за 24-бітною адресою 0x7FF000, після якої передано 16 тестових байтів:

```
03 0A 11 18 1F 26 2D 34 3B 42 49 50 57 5E 65 6C
```

3. *Опитування статусу (Polling):* циклічне читання регістра статусу командою 0x05. Зафіксовано стан зайнятості 0x03 (BUSY=1, WEL=1) до завершення внутрішнього циклу запису комірок зі скиданням у 0x00.
4. *Верифікація читанням:* виклик команди 0x03 (Read Data) з адреси 0x7FF000. Зчитані байти повністю ідентичні записаним, що доводить збереження цілісності даних.

3.1.5. Шина UART: Вивід системного логу прошивки (07_uart_hello_usart1.sr)

- На каналі D0 при швидкості 115200 бод (8N1) декодовано текстовий рядок:

```
\r\r\nHello!\r\n
```

Шістнадцятковий дамп: 0D 0D 0A 48 65 6C 6C 6F 21 0D 0A. Втрати байтів або помилок кадрування у штатному режимі не виявлено.

3.1.6. Методологія фіксації PWM та ADC логічним аналізатором

- **ШІМ (PWM):** вимірювання періоду T та ширини активного імпульсу t_{high} здійснюється безпосереднім підключенням каналу аналізатора до піна таймера. Для розрахунку коефіцієнта заповнення (Duty Cycle) використовуються два стани:
- 25% заповнення: $t_{high} = 0.25 \times T$.
- 75% заповнення: $t_{high} = 0.75 \times T$.
- **АЦП (ADC):** оскільки аналоговий рівень на логічному вході аналізатора викликає невизначеність порогів TTL/CMOS, момент та тривалість квантування фіксуються за допомогою службового цифрового виводу GPIO (Debug Pin):
- Прошивка встановлює пін у HIGH безпосередньо перед викликом HAL_ADC_Start().
- У тілі обробника переривання готовності даних HAL_ADC_ConvCpltCallback() або після закінчення блокуючого опитування прапорця EOC (End of Conversion) пін скидається в LOW. Тривалість імпульсу на аналізаторі точно відображає час перетворення АЦП мікроконтролера.

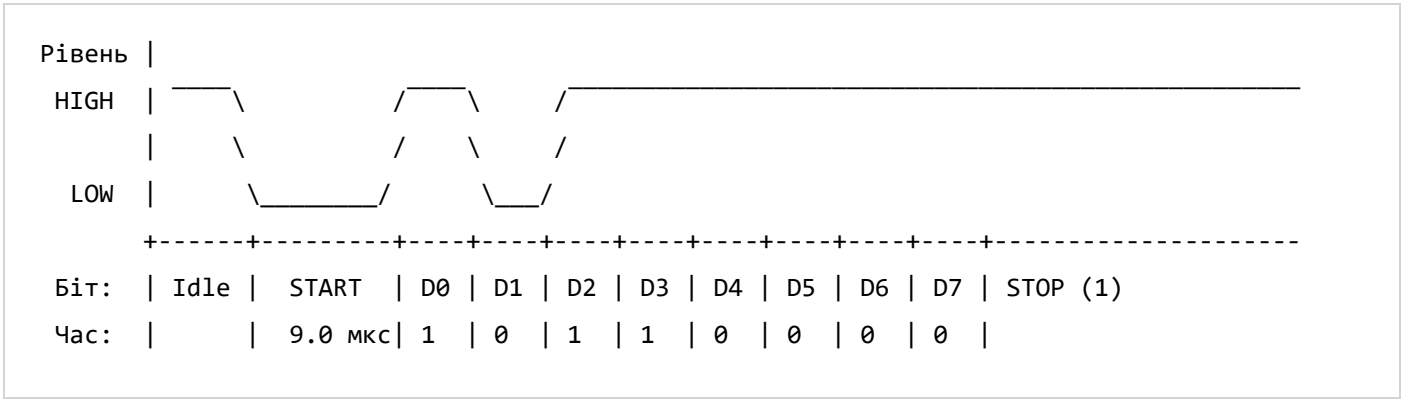
3.2 Аналіз часових параметрів та аномалії

Нижче наведена зведена таблиця часових параметрів інтерфейсів (табл. 10).

Файл сесії	Інтерфейс / Пристрої	Частота дискретизації, канали	Розрахункові значення	Фактичні виміряні значення (PulseView)	Відхилення / Примітка
`01_i2c_whoami.sr`	I²C: LSM303D, L3GD20H, BMP180	2 MS/s; `D0=SCL`, `D1=SDA`; тригер: спад `SDA`	$f = 100$ кГц, $T = 10$ μs	$f_{SCL} = 100.0$ кГц, $T_{SCL} = 10.00$ μs, тривалість транзакції ID: 194.0 μs	Повний збіг; міжбайтні розриви (Byte Gap) відсутні (0.0 мкс)
`02_i2c_data_bmp180.sr`	I²C: BMP180 (EEPROM + Дані)	2 MS/s; `D0=SCL`, `D1=SDA`	$T_{calib} \approx 2.27$ мс, $t_{conv} \geq 4.5$ мс	$T_{calib} = 2255.0$ μs, $t_{wait(T)} = 5.342$ мс, $t_{wait(P)} = 5.165$ мс	Запас очікування АЦП становить $+0.84$ мс та $+0.66$ мс; шина в паузах IDLE
`03_spi_jedec.sr`	SPI: Winbond W25Q64JV	2 MS/s; `D0=SCK`, `D1=MISO`, `D2=MOSI`, `D3=CS#`	SPI Mode 0; $f = 250$ кГц, $T = 4.0$ μs	$f_{SCK} = 250.0$ кГц, $T_{SCK} = 4.00$ μs	JEDEC ID вичитано без затримок у межах однієї транзакції CS#
`04_spi_flash.sr`	SPI: W25Q64JV (Page Program)	2 MS/s; ті самі	$T_{prog,\,nom} \approx 640$ μs, $t_{PP,\,typ} = 0.7$–3.0 мс	$\Delta t_{CS,\,PageProg} = 652.0$ μs, час запису комірок $t_{PP} \approx 1.45$ мс	Час програмування відповідає типовому діапазону datasheet Winbond
`05_sccb_read_pid.sr`	SCCB: OmniVision OV7670	1 MS/s; `D0=SIOC`, `D1=SIOD`	Програмний біт-бенгінг; $f \approx 10$ кГц	$f_{SIOC} = 10.0$ кГц, $T_{SIOC} = 100.0$ μs, тривалість читання ID: 1.97 мс	Двофазні цикли розділені апаратним `STOP`; Repeated START відсутній
`06_uart_selftest.sr`	UART: Self-test (Аномалія)	1 MS/s; `D0=TX`	115200 бод; $T_{bit} \approx 8.68$ μs	Початковий стан 0; $T_{imp} \approx 9.0$ μs; масові `Frame error`	**Аномалія:** старт захоплення під час ініціалізації піна GPIO
`07_uart_hello_usart1.sr`	UART: USART1 (Hello)	1 MS/s; `D0=TX`; тригер: спад `D0`	115200 бод; $T_{msg} \approx 954.8$ μs	$T_{msg} = 951.0$ μs, $T_{bit,\,факт} \approx 8.65$ μs (115668 бод)	Відхилення бітрейту $+0.41\%$ (у межах допустимого стандарту $\pm 2\%$)

3.3 Ручне декодування за формою сигналу

Для перевірки достовірності протокольного декодера проведено ручний побітовий аналіз першого байта пакета UART у файлі `07_uart_hello_usart1.sr` (байт `0x0D`, символ `\r`).



1. **Стан очікування (Idle):** до відмітки +93.0 µs+93.0 μs лінія утримується у високому рівні (1).
2. **Start-біт:** спадний фронт фіксує перехід у логічний 0 на інтервалі +93.0 µs⋯+102.0 µs+93.0 μs⋯+102.0 μs (тривалість ≈9 µs≈9 μs, що відповідає 1 бітовому інтервалу при 115200 бод).
3. **Шина даних (LSB First — від молодшого до старшого):**

- Біт 0: рівень 1 (вибірка на +106 µs+106 μs) →1→**1**
- Біт 1: рівень 0 (вибірка на +115 µs+115 μs) →0→**0**
- Біт 2: рівень 1 (вибірка на +124 µs+124 μs) →1→**1**
- Біт 3: рівень 1 (вибірка на +132 µs+132 μs) →1→**1**
- Біти 4..7: рівень 0 (тривала пауза низького рівня тривалістю ≈35 µs≈35 μs, тобто 4 біти підряд) →0,0,0,0→**0**,**0**,**0**,**0**

4. **Складання байта:**

Послідовність бітів: [D0…D7]=1,0,1,1,0,0,0,02 ⟶ Байт: 000011012=0x0D (символ CR).

Stop-біт: повернення лінії у високий рівень 1 на +170.0 µs+170.0 μs із тривалістю не менше одного бітового інтервалу. Ручний розбір повністю збігається з виводом протокольного декодера PulseView.

3.4 Дослідження аномалії: зрив синхронізації та помилки кадрування (06_uart_selftest.sr)

1. **Опис симптому:** під час спроби декодування сесії 06_uart_selftest.sr виникають масові попередження Frame error, а замість ASCII-рядка отримуються спотворені значення 0xC3, 0x43, 0xE1. Програмна інверсія каналу результату не дає.

2. **Аналіз першопричини (Root Cause):**

- **Перехідний стан лінії:** на початку таймлайну (t=0⋯+10 µs) лінія перебуває у низькому логічному рівні (0). Захоплення розпочалося до переведення виводу GPIO у режим Alternate Function Push-Pull з підтяжкою до живлення. Декодер сприйняв цей стартовий нуль як початок передачі (Start bit).
- **Фазовий зсув (Phase Accumulation):** при частоті дискретизації 1 MS/s дискрет вибірки становить 1.0 µs. За швидкості 115200 бод (Tbit ≈ 8.68) на

один біт припадає лише 8 або 9 семплів. Накопичена помилка квантування призвела до стробування сигналу на фронтах переходів.

- **Наслідок:** вибірка стоп-біта припала на ділянку низького рівня замість високого, що згенерувало апаратну помилку Framing Error (порушення формату кадру).

3. **Рекомендація щодо усунення:** налаштовувати апаратний тригер аналізатора на спадний фронт (Falling Edge) із попередньою умовою стабільного високого рівня (High Level Pre-trigger) для відсікання перехідних процесів запуску мікроконтролера.

3.5 Обмеження захоплення паралельної шини камери (DVP: VSYNC, HREF, PCLK)

Згідно з технічними умовами підключення сенсора OV7670, передача відеопотоку здійснюється паралельним інтерфейсом DVP:

- **Частота піксельного тактування (PCLK):** при роздільній здатності VGA/QVGA становить від **12 до 24 МГц**.
- **Кадрова та рядкова синхронізація:** VSYNC (≈ 30 Гц), HREF (рядки зображення).

Апаратні обмеження аналізатора:

- Використаний 8-канальний аналізатор Saleae/FX2LP має максимальну межу дискретизації **24 MS/s** при передачі через шину USB 2.0 (High-Speed), а стабільний неперервний потік для 8 каналів часто обмежується **12–16 MS/s**.
- Відповідно до теореми Найквіста-Котельникова, для реєстрації сигналу частотою 24 МГц необхідна частота дискретизації щонайменше 48 MS/s, а для достовірного вимірювання форми імпульсів і часових параметрів setup/hold — не менше 120–240 MS/s(у 5–10 разів вище тактової).

- Пряме захоплення сигналу PCLK на повному відеопотоці цим аналізатором призводить до накладання спектрів (аліасингу) та переповнення буфера FIFO.

Спосіб перевірки:

1. *Низькошвидкісні сигнали на логічному аналізаторі:* контролювати виключно сигнали VSYNC та HREF, налаштувавши попереднє зниження частоти тактування сенсора через внутрішній переддільник регістру CLKRC (Prescaler).
2. *Верифікація PCLK на реальному обладнанні:* проводити аналіз форми піксельного такту та часових затримок між даними D[7:0] і PCLK виключно за допомогою широкосмугового двоканального осцилографа (із смугою пропускання від 100 МГц) або високошвидкісного логічного аналізатора класу 100+ MS/s.

3.6 Перелік обов'язкових артефактів Напряму 4

Усі артефакти розміщено у відповідних каталогах репозиторію проєкту:

- **Файли сесій логічного аналізатора:** docs/captures/01_i2c_whoami.sr, 02_i2c_data_bmp180.sr, 03_spi_jedec.sr, 04_spi_flash.sr, 05_sccb_read_pid.sr, 06_uart_selftest.sr, 07_uart_hello_usart1.sr.
- **Текстові анотації декодування (sigrok-cli):** docs/captures/annotations/01_i2c_whoami.txt, 02_i2c_data_bmp180.txt, 03_spi_jedec.txt, 04_spi_flash.txt, 05_sccb_read_pid.txt, 06_uart_selftest.txt, 07_uart_hello_usart1.txt.
- **Графічні осцилограми з вимірювальними маркерами:** docs/lab_2/img/01_i2c_period.png, 01_i2c_transaction.png, 02_i2c_bmp180.png, 03_spi_jedec.png, 04_spi_flash_program.png, 05_sccb_pid.png.

- **Аналітичний звіт та тестові вектори:** docs/captures/analysis.md (включає калібрувальну матрицю BMP180, тестовий патерн W25Q64JV та вектори для хостових модульних тестів за Контрактом 4 → 6).

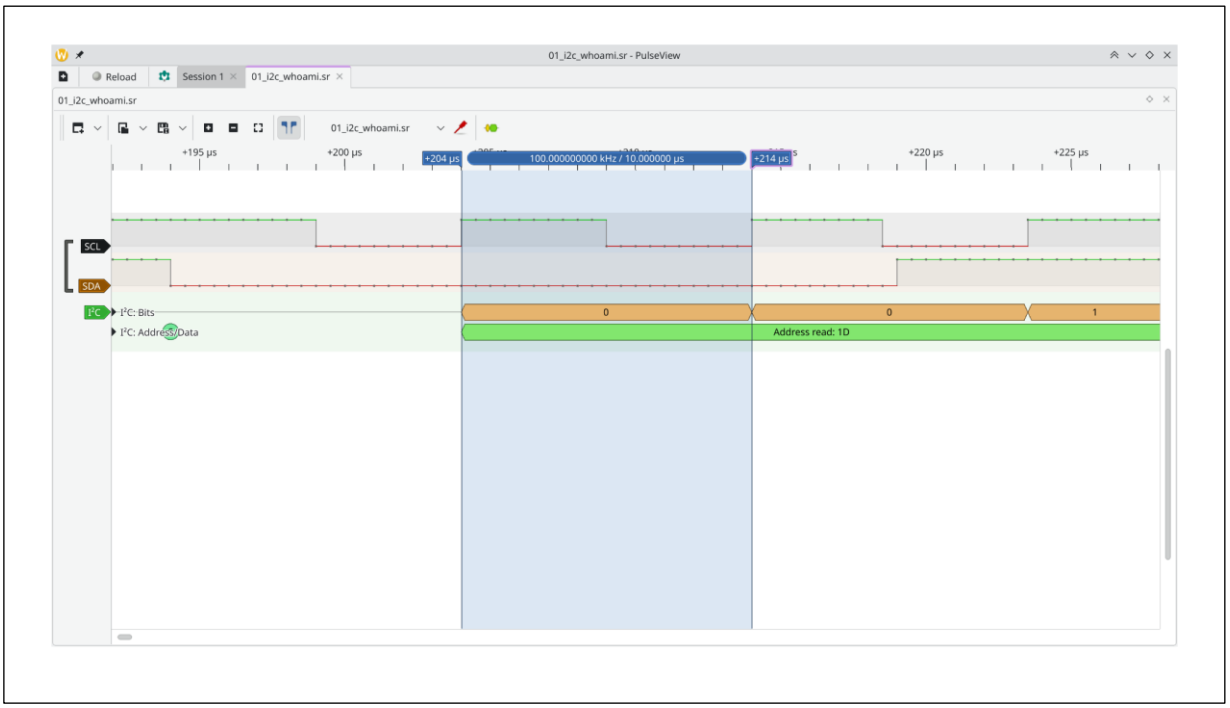


Рисунок 2 — 01_i2c_period



Рисунок 3 — 01_i2c_transaction



Рисунок 4 — 03_spi_jedec

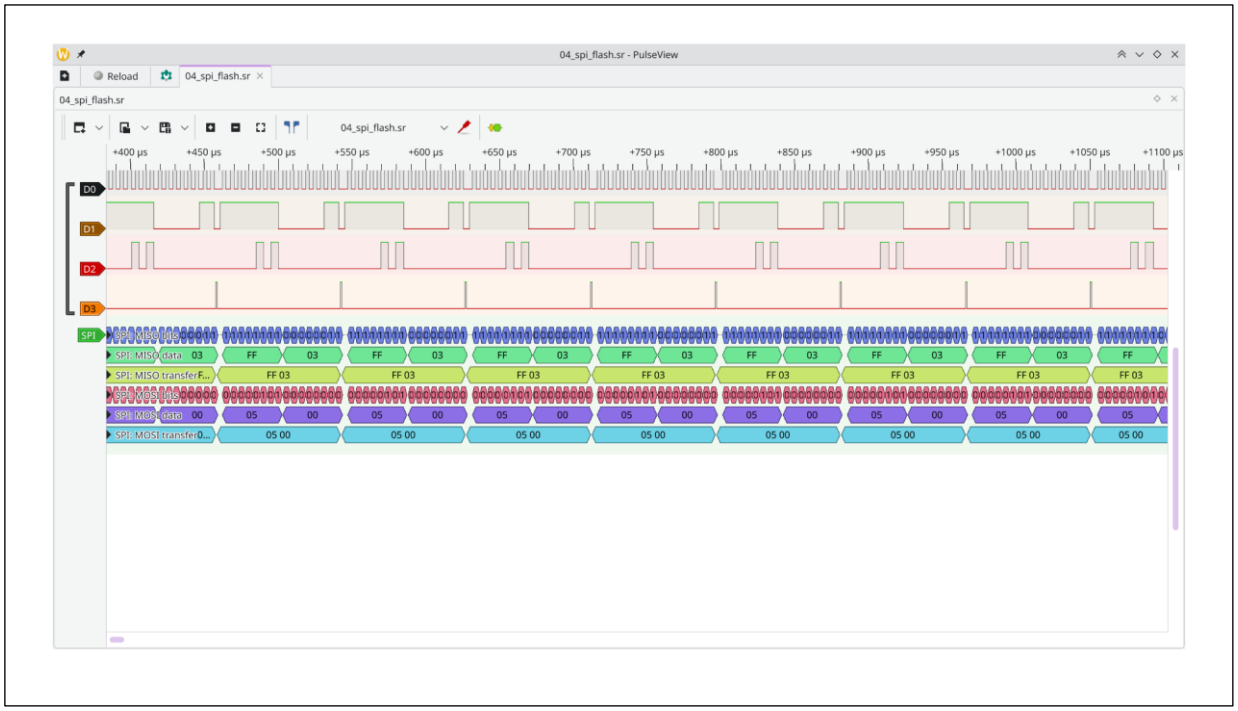


Рисунок 5 — 04_spi_flash_program

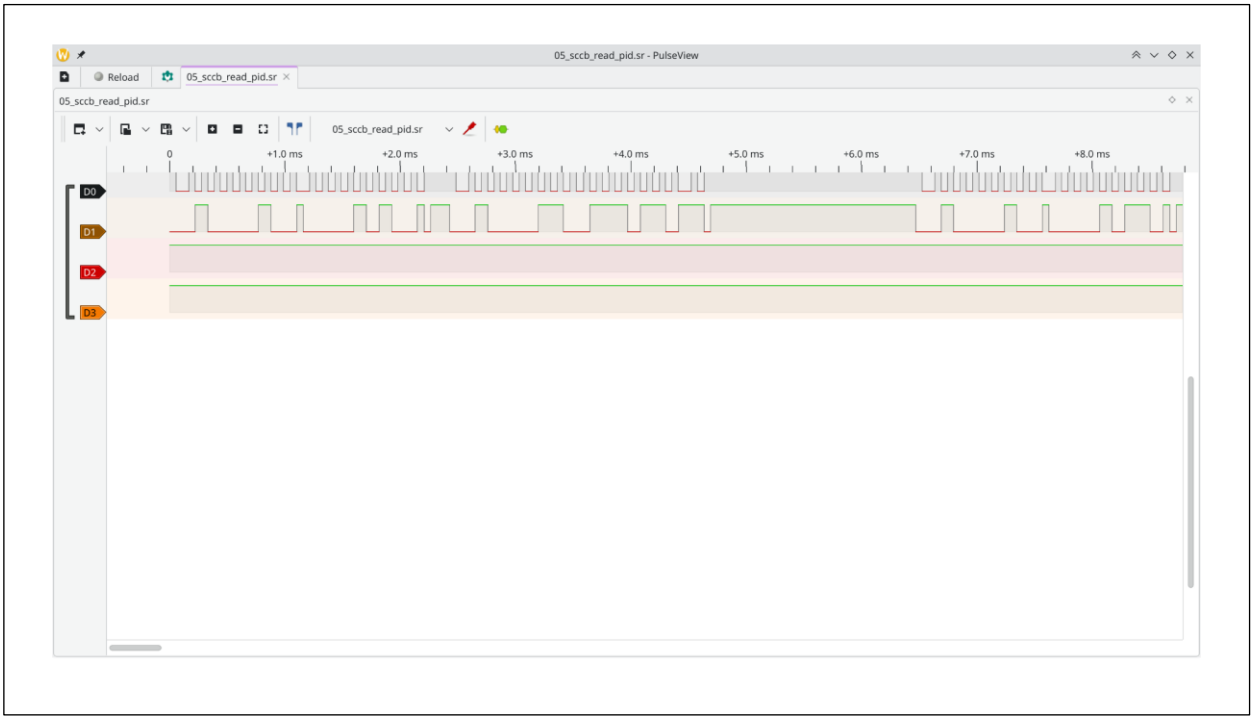


Рисунок 6 — 05_sccb_pid

4. Архітектура програмного забезпечення та система збірки

Етап 1. Проєктування та розробка системи Make-файлів

На першому етапі було створено дворівневу систему Makefile для автоматизації збірки прошивки та розділення завдань між модулями.

1. **Розробка внутрішнього firmware/Makefile:**
    - **Налаштування тулчейну:** Вказано крос-компілятор arm-none-eabi-gcc та задано прапорці архітектури ядра ARM Cortex-M4 з апаратним FPU (-mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard).
    - **Підключення модулів:** Прописано шляхи до заголовочних файлів усіх шарів проєкту (-ICore/Inc, -IApp, -IPlatform, -IDrivers/...), що дозволило пов'язати низкорівневий HAL, адаптери шин та вищу логіку.
    - **Контроль якості (Quality Gate):** Додано строгі прапорці компіляції -Wall -Wextra -Og -g, які змушують компілятор перевіряти сумісність типів та шукати потенційні помилки.

- **Автоматизація артефактів:** Налаштовано автоматичну генерацію файлів залежностей (.d), комбіноване лінкування у firmware.elf та конвертацію у вихідні формати .hex і .bin з виводом розміру пам'яті через arm-none-eabi-size.

2. **Створення кореневого ./Makefile (обгортки):**
   - Створено глобальний Makefile, який делегує виконання команд clean та all усередину ізольованого Docker-контейнера.
   - Налаштовано автоматичне монтування каталогу проєкту (-v $(pwd):/workspace) та передачу ID користувача (--user 1000:1000), щоб згенеровані у папці build/ файли мали правильні права доступу на хост-системі.

Етап 2. Контейнеризація середовища збірки (Docker)

Щоб збірка не залежала від конфігурації конкретного ПК і гарантовано відтворювалася на будь-якій системі, було налаштовано Docker-середовище.

1. **Конфігурація docker/Dockerfile:**
   - Базовим образом обрано ubuntu:22.04 у неінтерактивному режимі.
   - Автоматизовано встановлення необхідних інструментів розробки: gcc-arm-none-eabi, binutils-arm-none-eabi, libnewlib-arm-none-eabi, make, python3, git, stlink-tools тощо.

2. **Створення локального образу:**
   - Зібрано та затеговано локальний Docker-образ stm32-build, який використовується кореневим Makefile для всіх операцій збірки.

Етап 3. Інтеграція змін та підготовка до тестування (Етап 4)

Після завершення суміжних етапів колегами (підтягування тестових векторів і розрахунків) було виконано синхронізацію та підготовку робочого середовища.

1. **Синхронізація репозиторію:**

- Отримано останні зміни з віддаленої гілки origin/main.
- Розв'язано конфлікти та відхилення гілок у локальному середовищі, зведено стан коду до актуального фінального комміту.

2. **Перевірка наявності та шляхів до конфігураційних файлів:**
   - Перевірено та підтверджено структуризацію файлів у репозиторії (docker/Dockerfile, firmware/Makefile, кореневий Makefile).

Етап 4. Фінальне локальне тестування та перевірка в CI/CD

На заключному етапі було проведено повний цикл збірки й верифікації проєкту на дотримання вимог якості.

1. **Запуск чистої збірки у Docker:**
   - Виконано команду make clean && make -j$(nproc), яка очистила попередні бінарники та провела повну збірку всіх шарів проєкту всередині контейнера stm32-build.

2. **Аналіз логів компіляції (-Wall -Wextra):**
   - Перевірено вивід компілятора при збірці модулів main.c, bringup.c, bus_adapters.c та HAL-драйверів.
   - **Результат:** Проєкт скомпільовано повністю без жодних попереджень (0 warnings).

3. **Контроль згенерованих артефактів:**
   - Успішно зібрано підсумковий файл firmware.elf та згенеровано бінарні файли прошивки .hex і .bin.

4. **Перевірка у GitHub Actions:**
   - Перевірено статус автоматичного пайплайну в GitHub Actions - віддалена збірка та хостові тести проходять успішно зі статусом Success.

5. Автоматизоване тестування програмного забезпечення

Відповідальний: інженер з тестування (напрям 6). Артефакти: каталог tests/, команди make test і make coverage, звіт покриття docs/test-coverage.md.

5.1. Призначення та підхід

Логіка застосунку й драйверів сенсорів (перетворення сирих даних у фізичні величини, ідентифікація чіпів, робота з SPI-flash, розбір UART-команд) перевіряється на хості звичайним компілятором gcc, без плати. Це можливо, тому що код працює з шинами не напряму через HAL, а через структури з покажчиками на функції (g_i2c_platform_bus, g_spi_platform_bus, g_sccb_platform_bus, інтерфейс firmware/Platform/bus_interface.h). У прошивці їх реалізує Platform/bus_adapters.c поверх HAL, у тесті — fake-реалізація tests/fakes/fake_bus.c.

Код у Core/ (ініціалізація периферії CubeMX) і HAL не тестується на хості: він залежить від регістрів МК і містить мало прикладної логіки. Основним об'єктом тестування є драйвери та прикладна логіка, бо саме в них розрахунки, обробка помилок і послідовності команд.

5.2. Запуск тестів без плати

Потрібні gcc, make і gcov. З кореня репозиторію:

```
make test       # запуск тестів (код 0 = усі пройшли)
make coverage   # те саме + покриття і перевірка порогу
```

Вивід printf прошивки під час тестів зберігається в tests/build/firmware_output.log. У CI обидві команди виконуються окремим job test у GitHub Actions на кожен pull request.

5.3. Тестовий каркас і test doubles

Таблиця 11

Елемент	Вид double	Призначення
tests/minitest.h	фреймворк	Мінімальний каркас: макроси CHECK, CHECK_EQ, KNOWN_DEFECT, RUN. Не зупиняється на першій помилці, рахує перевірки, повертає ненульовий код при провалі.
fake I²C	fake + журнал	Регістрова модель LSM303D, L3GD20H, BMP180 (після команди 0x2E/0x34 віддає UT/UP). Журнал транзакцій (пристрій, регістр, довжина, дані). Ін'єкція збоїв: номер транзакції, що повертає BUS_ERROR, або адреса, що не відповідає (NACK).
fake SPI (W25Q64)	fake + mock	Модель флешу 8 МіБ: JEDEC, WREN/WEL, стирання сектора, запис сторінки, читання, біт BUSY. Журнал команд (код, адреса, довжини) дозволяє перевіряти порядок як у mock; лічильники записів без WREN і звернень при CS=HIGH.
fake SCCB, fake UART	fake	Регістри OV7670 (PID 0x76, VER 0x73) з режимом збою; черга вхідних і буфер вихідних байтів USART1.

Відмінність видів doubles. Fake — спрощена працююча реалізація (модель чіпа в пам'яті). Stub — повертає заздалегідь задану відповідь, наприклад BUS_ERROR у режимі збою. Mock — перевіряє очікувані виклики та їх порядок; у нас це журнал транзакцій fake-шин. Саме mock-перевірка потрібна для послідовності команд SPI-flash: результат функції сам не показує, чи було подано WREN перед PAGE PROGRAM.

Власний фреймворк, він не дає автоматично згенерованих mock-ів, розширених порівнянь типів і готових звітів; натомість порядок транзакцій перевіряється через журнали fake-шин.

5.4. Що перевіряється

Таблиця 12

Вимога методички	**Тести**	**Результат**
Перетворення сирих даних у фізичні одиниці	LSM303D (0,061 мг/LSB: 16384 → 999 мг, −32768 → −1998 мг, 32767 → 1998 мг); L3GD20H (8,75 мдпс/LSB: 256 → 2240 мдпс, −1 → −8); розбір big-endian (be16); BMP180.	пройдено
Контрольний приклад datasheet	BMP180 (Bosch): коефіцієнти AC1…MD, UT = 27898 → T = 15,0 °C; UP = 23843 → P = 69964 Па.	пройдено
Обробка помилок шини: NACK	Пристрій, що не відповідає (LSM303D, повна послідовність read_xyz); selftest повертає FAIL.	пройдено
Обробка помилок шини: збій / timeout	Збій кожної читальної транзакції BMP180; збій читання XYZ (вихідні значення не псуються); збій SPI-обміну (CS відпускається). Окремого коду timeout у bus_status_t немає.	пройдено, часткове
Неочікуваний ID	LSM303D (0x49), L3GD20H (0xD4), BMP180 (0x55), JEDEC W25Q64 (EF 40 17), OV7670 PID/VER (0x76/0x73): правильні та помилкові значення.	пройдено
Послідовність команд SPI-flash	WREN → SECTOR ERASE (0x20, 0x7FF000) → опитування SR1 → WREN → PAGE PROGRAM (16 байтів) → опитування SR1 → READ DATA; на шині немає зайвих команд; fake ігнорує запис без WREN (лічильник = 0); очікування BUSY; виявлення спотворених даних.	пройдено
UART-команди	echo, невідома команда, переповнення буфера, id, i2c, data, spi, flash, sccb, uart, help, порожній рядок, ініціалізація.	пройдено
Реальні дані логічного аналізатора	Далі описано	пройдено

Усього 35 тестів: перетворення даних — 5, послідовність BMP180 і помилки запису — 3, помилки шин — 3, ідентифікація — 4, SPI-flash — 4, UART — 11, вектори з аналізатора — 5.

5.5. Тестові вектори з реальних записів логічного аналізатора

Джерело: docs/captures/analysis.md, декодування *.sr у PulseView. Очікувані значення обчислено окремим розрахунком за формулами Bosch , а не взято з виводу самої прошивки, щоб перевірка не замикалась сама на себе.

Таблиця 13

Запис	Байти на шині	Результат драйвера	Збіг
02 (BMP180)	22 байти калібрування (19 9C FB 72 … 0A 94), UT = 0x54BB (21691), UP = 0x9FD6 (40918)	T = 23,7 °C (237), P = 99817 Па (998,17 гПа); коефіцієнти збігаються з таблицею аналітика	так
01, 02 (LSM303D)	3F FE F8 7F 9B 43; 4D FE F8 7F 8E 43	(−27; 1998; 1055) мг; (−26; 1998; 1054) мг	так
04 (flash)	16 байтів 03 0A 11 … 6C за адресою 0x7FF000	прошивка записує рівно ці байти (шаблон i·7+3)	так
03 (JEDEC)	MISO: EF 40 17	мануфактурер 0xEF, тип 0x40, ємність 0x17	так
07 (UART)	0D 0D 0A 48 65 6C 6C 6F 21 0D 0A	вивід команди uart збігається побайтно	так

Спостереження за векторами. (1) Тиск за розрахунком 998,17 гПа, а в analysis.md вказано ≈998,4 гПа; тест допускає ±0,5 гПа. (2) Модуль прискорення з записів 01 і 02 ≈ 2,26 g замість ≈ 1 g (вісь Y = 0x7FF8, майже повна шкала ±2 g); арифметика прошивки правильна, тому потрібна перевірка стенда. У тесті це позначено як KNOWN_DEFECT.

5.6. Результати виконання

Таблиця 14

Показник	Значення
Кількість тестів / перевірок	35 / 216
Провалено	0
Відомі дефекти (KNOWN_DEFECT, не провал)	4
Попередження компіляції (-Wall -Wextra)	0
Локальний запуск	Windows, Git Bash, gcc 15.2.0: make test → код 0; make coverage → поріг досягнуто
CI	GitHub Actions, pull request #7 (гілка tests → main): jobs build і test успішні

6. Хід виконання та журнал проблем

6.1. Організація та хронологія виконання лабораторної роботи

Роботу продовжено в спільному репозиторії, створеному під час ЛР № 1. Вихідними матеріалами стали PRD мультирежимного реєстратора інцидентів, опис апаратних компонентів і працездатний конвеєр збірки та прошивки. Нижче наведено послідовність виконання робіт і контракти між учасниками.

Таблиця 15

Етап	Відповідальні	Зміст робіт	Передавання результату / контракт
1. Уточнення вихідних вимог	Системний аналітик і технічний письменник; команда	На основі PRD та переліку компонентів підтверджено цільову плату NUCLEO-F411RE і склад стенда: GY-89, OV7670, W25Q64, YP-05. Визначено інтерфейси, які потрібно розвести одночасно.	PRD і hardware-components.md є вихідними джерелами. Будь-яка зміна складу пристроїв має бути погоджена до завершення pinout.
2. Дослідження документації та підготовка pinout	Системний аналітик і технічний письменник	Опрацьовано datasheet мікроконтролера й сенсорів, RM0383 та UM1724; зіставлено alternate functions, контакти CN7/CN10, адреси, ID-регістри, живлення, підтяжки та маршрути DMA. Підготовлено pinout.md.	Чернетка pinout передається інженеру апаратного забезпечення до монтажу, а архітектору ПЗ — для перевірки відповідності .ioc та програмній конфігурації.

Етап	Відповідальні	Зміст робіт	Передавання результату / контракт
3. Перевірка можливості апаратної реалізації	Інженер апаратного забезпечення; архітектор вбудованого ПЗ; аналітик протоколів	Перевірено, чи доступні потрібні сигнали на корпусі та роз'ємах плати, чи сумісні вибрані AF і DMA-маршрути, чи можна незалежно користуватися I2C1, SCCB, SPI2 та UART. Виявлені розбіжності мають бути зафіксовані в pinout, а не залишатися лише на макетній платі.	Контракт pinout ↔ .ioc: кожен фізичний сигнал має відповідати документації й конфігурації прошивки. Після зміни піна оновлюються таблиця, схема та програмна конфігурація.
4. Монтаж і початкова перевірка	Інженер з апаратного забезпечення	Після погодження розкладки стенд збирається без пайки. Перевіряються живлення й спільна земля, доступність пристроїв, читання ID-регістрів і отримання правдоподібних даних у єдиній прошивці.	Передаються фото стенда, результати вимірювання живлення та UART-лог. Якщо контакт або сигнал відрізняється від pinout, фіксується конкретний пін, симптом і причина до зміни схеми.
5. Запис і декодування сигналів	Інженер з апаратного забезпечення; аналітик протоколів	На кожний фактично використаний інтерфейс готуються відтворювана транзакція, файл запису та параметри захоплення. Аналітик декодує сигнали в PulseView, вимірює часові характеристики й зіставляє їх із документацією.	Контракт апаратний напрям → аналіз протоколів: разом із .sr передаються частота дискретизації, тригер, призначення каналів, команда прошивки для повторення запису та очікуваний зміст транзакції.
6. Архітектура та тестування ПЗ	Архітектор вбудованого ПЗ; інженер з тестування	Код організовується за шарами; драйвери взаємодіють із шинами через погоджені інтерфейси, а бізнес-логіка не залежить від HAL. На хості готуються тести перетворення даних, обробки помилок і послідовності команд SPI Flash.	Контракт архітектор → тестувальник: до написання тестів узгоджується API драйверів, типи помилок і спосіб підміни шини. Аналітик протоколів передає декодовані байти як тестові вектори.
7. Інтеграція звіту та фінальна звірка	Системний аналітик і технічний письменник; усі учасники	Матеріали напрямів збираються в docs/lab2-report.md: pinout, стенд і вимірювання, аналіз захоплень, архітектура, тести, проблеми, внесок і джерела. Перевіряється однаковість назв сигналів, адрес, частот і шляхів до артефактів.	Кожен учасник передає результат у формі таблиці, логів, скриншотів, посилань на коміти або PR.

Як ми можемо бачити з вищенаведеної таблиці, робота виконувалася за ланцюгом залежностей: pinout і вимоги → монтаж та конфігурація ПЗ → записи сигналів → декодування й тестові вектори → інтегрований звіт. Частина напрямів могла працювати паралельно, але межі між ними задавалися артефактами й інтерфейсами, що зменшувало ризик неузгоджених пінів, протоколів або API.

6.2. Журнал проблем

Окрім тих проблем, які були описані у розділах звіту, наша команда зіштовхнулась із проблемами, які наведені в табл. 16.

Таблиця 16

№ / проблема	Симптом	Першопричина (root cause)	Виправлення та підтвердження
1. Неоднозначність формату адрес I²C/SCCB	Під час підготовки таблиці адрес виникла неоднозначність: для OV7670 в документації трапляються байти 0x42 (запис) і 0x43 (читання), тоді як адреса пристрою в 7-бітному форматі дорівнює 0x21. Аналогічно для сенсорів треба було відрізняти 7-бітну адресу від байта, що передається на шині.	У документації SCCB адресні байти подано разом із бітом R/W, а в STM32 HAL для I²C параметр DevAddress задається у форматі addr7 << 1. Змішування цих представлень могло призвести до помилкової адресації та NACK під час bring-up.	У pinout адреси рознесено за форматами: 7-бітна адреса; байт запису; байт читання. Для I²C окремо зафіксовано правило передавання addr7 << 1 у HAL. Для OV7670 наведено 0x21 та байти 0x42/0x43. Очікувані ID-регістри описано як критерій перевірки, а остаточні значення мають бути зіставлені з реальною відповіддю пристрою.
2. Розбіжність між характеристиками мікросхем і готового модуля GY-89	Під час вибору живлення та підтяжок з'ясувалося, що datasheet-и LSM303D, L3GD20H і BMP180 описують самі мікросхеми, тоді як готовий GY-89 може містити регулятор, перетворювачі рівнів та підтягувальні резистори. Тому номінали й межі живлення окремої IC не можна автоматично переносити на весь модуль.	Використовуються джерела різних рівнів: datasheet кристала, специфікація breakout-модуля та фактичне виконання конкретної плати GY-89. Їх змішування могло призвести до неправильного вибору VIN/3V3 або до додаткових паралельних підтяжок на I²C.	У pinout окремо вказано живлення готового GY-89 через VIN згідно з документацією використаного модуля та позначено, що основні pull-up мають бути на модулі/шині, а внутрішні pull-up STM32 не використовуються як заміна. Споживання готового модуля позначено як проєктний резерв, який необхідно підтвердити вимірюванням; перед увімкненням стенда слід перевірити конкретну ревізію модуля й логічні рівні SDA/SCL.
3.Неможливість записати SPI на високій частоті	При захопленні на частоті дискретизації понад 2 МГц PulseView не починав запис: програма вимикалась або вилітала через досить короткий момент часу, тож зафіксувати сигнали було неможливо.	Згідно pinout.md, частота SCK SPI2 дорівнює 3.125 МГц (дільник 16), для її запису потрібна частота дискретизації вища за 2 МГц. Наявний аналізатор з PulseView на таких частотах працював нестабільно.	У CubeMX для SPI2 збільшено Baud Rate Prescaler з 16 до 256. Частота SCK знизилась з 3.125 МГц до ≈ 195 кГц. І запис виконано на частоті 2 МГц.
4.Хостові тести не збиралися на Windows	make test завершувався помилкою CreateProcess(... cc ...) failed, system cannot find the file specified; тести не запускалися.	На комп'ютері не було компілятора gcc у PATH, а в tests/Makefile рядок CC ?= gcc не перебивав вбудоване значення CC=cc, тому make шукав cc.	Встановлено gcc через scoop; у tests/Makefile змінено на CC := gcc. Підтвердження: make test дає 216 перевірок, 0 провалених, код 0; job test у CI зелений.

Висновки:

У результаті виконання лабораторної роботи ми опрацювали основні етапи інтеграції сенсорної підсистеми мультирежимного реєстратора інцидентів на базі STM32 NUCLEO-F411RE. Основну увагу було приділено проєктуванню схеми підключення компонентів, вибору виводів мікроконтролера, перевірці сумісності апаратних ресурсів і визначенню параметрів периферійних інтерфейсів. На основі документації виробників розроблено pinout-таблицю для модулів GY-89, OV7670, W25Q64 та USB-UART адаптера, визначено адреси й очікувані ідентифікатори пристроїв, а також опрацьовано питання живлення, тактування та використання DMA. Окремо було розглянуто обмеження STM32F411RE під час роботи з паралельним інтерфейсом камери, оскільки цей мікроконтролер не має апаратного блока DCMI.

У процесі роботи ми поглибили розуміння того, що правильне підключення периферійних пристроїв залежить не лише від призначення окремих GPIO, а й від сумісності альтернативних функцій, доступності каналів DMA, особливостей штатної обв’язки Nucleo та електричних характеристик модулів. Також було визначено порядок перевірки пристроїв за їхніми ідентифікаційними регістрами, сформовано параметри основних інтерфейсів і підготовлено основу для верифікації сигналів логічним аналізатором. Важливим результатом стало розуміння необхідності зіставляти програмні повідомлення з фактичними сигналами на фізичних лініях, щоб виявляти помилки підключення, налаштування та передавання даних.

Окрім апаратної частини, лабораторна робота передбачала підготовку до впорядкування програмного проєкту, організації хостового тестування та подальшої інтеграції результатів у спільний репозиторій. Розподіл відповідальності між учасниками та визначення контрактів передавання результатів дали змогу розділити роботу над документацією, монтажем, аналізом протоколів, архітектурою програмного забезпечення і тестуванням. Фіксація технічних рішень,

проблем і джерел у документації створює основу для відтворюваності стенда та полегшує подальше налагодження.

Результати, одержані в ході виконання цієї роботи, формують програмно-апаратне підґрунтя для наступних лабораторних робіт, у яких основна увага приділятиметься реалізації режимів Cargo і Site, виявленню інцидентів, збереженню журналу та отриманню зображень за відповідними подіями.

Список використаних джерел:

1. STMicroelectronics. STM32F411xC/E — Datasheet DS10314
2. STMicroelectronics. STM32F411xC/E — Reference Manual RM0383
3. STMicroelectronics. STM32 Nucleo-64 boards — User Manual UM1724
4. GY-89 — опис готового модуля (Beemong)
5. GY-89 — опис готового модуля (Arduino.ua)
6. STMicroelectronics. LSM303D — Datasheet
7. STMicroelectronics. LSM9DS0 — Datasheet
8. STMicroelectronics. L3GD20H — Datasheet
9. Bosch Sensortec. BMP180 — Datasheet
10. OmniVision OV7670 — Datasheet (копія, розміщена на сайті Olimex).
11. Arducam. OV7670 CMOS Camera Module — Module Documentation.
12. OmniVision. OV7670/OV7171 CMOS VGA CameraChip Implementation Guide.
13. OV7670 — Datasheet V1.4, copy/reference PDF.
14. MIT 6.111. OV7670 application/implementation notes.
15. Winbond. W25Q64FV — manufacturer documentation.
16. W25Q64FV — datasheet copy/reference PDF.
17. FTDI. FT232R — Datasheet.
18. GitHub. GitHub Actions — official documentation.
19. Docker. Dockerfile reference — official documentation.
20. Arm. Arm GNU Toolchain — compiler and tools for embedded development.
21. picocom — terminal emulator for serial interfaces.
22. sigrok. PulseView — logic analyzer and protocol decoder interface.

23. sigrok. Getting started with a logic analyzer.

24. ThrowTheSwitch. Unity — unit test framework for C.

25. ThrowTheSwitch. CMock — mock generation for C.

26. ThrowTheSwitch. Ceedling — build and test system for embedded C.